

AI-Driven Network Attack Forecasting via World Models

Technical Deep-Dive & Engineering Specification

Challenge Theme: NTRO Blockchain & Cybersecurity (Track 2) | **Execution Target:** Edge Tactical Node (16GB RAM / 4GB VRAM)

1. Executive Summary & Core Results

Modern enterprise and critical infrastructure defense is crippled by the **temporal blindspot** of static intrusion detection systems. Signature engines and flow classifiers inspect traffic in isolation after packets have already transited the perimeter. This project conceives, trains, and operationalizes an **AI Network World Model** that learns the underlying environmental transition dynamics $P(S_{t+1} | S_t)$ of telemetry states. Operating on 2.0-second time windows across a 22-dimensional dual-level feature space (NetFlow statistics and packet-level micro-behaviors), our World Model autoregressively simulates $K = 5$ steps into the future (up to +10.0 seconds), forecasting multi-stage intrusion trajectories before compromise execution.

Performance Dimension	Empirical Finding (Held-Out Test Set)	Operational Significance
Binary Infiltration F1	0.7153 (World Model) vs 0.5479 (Baseline LR)	+30.6% genuine F1 gain with high precision (63.4% vs 39.3%)
False Positive Rate (FPR)	12.87% (World Model) vs 37.98% (Baseline LR)	66.1% reduction in raw false alarms (Baseline flags 38% of benign!)
Same-Threshold F1 (tau=0.50)	0.6789 (World Model) vs 0.5479 (Baseline LR)	World Model cuts baseline FPR in half (19.87% vs 37.98%) at equal recall
Threat Forecast Lead Time	1.50 seconds average advance warning	Enables automated firewall rule staging prior to payload completion
End-to-End Pipeline Latency	10.20 ms (Mean) / 10.11 ms (Median)	Consumes only 0.51% of the 2.0-second window cadence budget
Local Hardware Footprint	13.9 MB (Redis) + 783 MB (Python Stack)	Runs 100% offline on a tactical laptop with >3.3 GB RAM headroom

2. Problem Framing: Why World Models Over Static Classifiers

Traditional Intrusion Detection Systems (NIDS) frame attack detection as an instantaneous mapping from feature vector to label: $X_t \rightarrow y_t \in \{0, 1\}$. In security operations, static models that optimize for high recall (~90%) inevitably suffer from catastrophic false-positive rates (here, **37.98% FPR**), overwhelming analysts with hundreds of alerts per hour. Intrusions are not stationary stochastic processes; they are structured, multi-phase Markov-like trajectories unfolding across time: **Reconnaissance (TA0043) → Initial Access (TA0001) → Lateral Movement (TA0008) → Command & Control (TA0011)**.

To solve this, we adapt the **World Model paradigm** (Ha & Schmidhuber, 2018) to network telemetry. The model learns an internal representation of network state dynamics: given a 10-step history ($S_{t-9} \dots S_t$), it predicts the next environment state S_{t+1} and recursively feeds predictions into its recurrent hidden state to rollout future trajectory horizons up to S_{t+K} .

3. Full System Architecture & '2-in-1' Shared Engine Design

- The system enforces a strict decoupled micro-pipeline where telemetry ingestion, state inference, persistent audit logging, and visualization operate through lightweight, standardized protocols without enterprise orchestration bloat:
- **Packet & Flow Ingestion:** Scapy and NetFlow parsers extract 22-dimensional features over 2.0-second discrete window frames.
 - **Local Redis Streams (network:telemetry:windows):** Decouples feature extraction from deep learning inference. Redis Streams provides millisecond pub/sub buffering with consumer groups and zero packet-drop guarantees.
 - **Unified Inference Engine (inference/engine.py):** A deliberate '2-in-1' architectural decision. Both the Terminal CLI and the Web SOC Dashboard bind to this single engine instance, guaranteeing zero discrepancy in threshold calibration, normalization scaling, or rollout simulation.
 - **Autoregressive Rollout Engine (model/rollout.py):** Simulates $K = 5$ forward steps (10 seconds), outputting future infiltration probabilities and anticipated MITRE ATT&CK stages.
 - **FastAPI Serving Layer:** Provides high-throughput REST endpoints (/api/analyze, /api/history) and an asynchronous WebSocket hub (/ws/live).
 - **SQLite Persistent Storage (storage/db.py):** Replaces heavy time-series databases (InfluxDB/TimescaleDB) with a single-file SQLite database indexed on timestamps, capturing historical forecasts for forensic audit trails.
 - **Dual Frontends:** (1) Hacker-Style Terminal Command Console (Rich/Textual TUI) for headless deployments; (2) React/Vite Dark-Mode Web SOC Dashboard with Recharts dual-curve trajectory plots.

4. Data Engineering: Multi-Scenario Real CTU-13 Telemetry

To guarantee high scientific validity, **zero synthetic or generated data was used**. The pipeline ingests authentic, full-size network captures from the Czech Technical University CTU-13 Botnet dataset, combining both packet-level PCAP and flow-level NetFlow streams:

Scenario & File	Raw Size	Telemetry Records	Dominant Attack Stages
CTU-13 Scenario 10 (Rbot) scen10_rbot.binetflow	308 MB	1,309,791 flows	Command & Control (C2) beaconing, ICMP flooding
CTU-13 Scenario 2 (Neris) scen2_neris_full.pcap	34.58 MB	176,000 packets	Reconnaissance port scanning, Initial Access probes
CTU-13 Scenario 2 (Neris) ctu_botnet_flow.netflow	3.16 MB	15,000+ flows	Lateral Movement, SPAM execution

Aggregated Class Distribution (14,890 Total 2.0-Second Windows):

- **Stage 0 (Benign):** 11,713 windows (78.6%) |
- **Stage 1 (Reconnaissance):** 1,755 windows (11.8%) |
- **Stage 2 (Initial Access):** 172 windows (1.15%) |
- **Stage 3 (Lateral Movement):** 1,118 windows (7.5%) |
- **Stage 4 (Command & Control):** 142 windows (0.95%).

All 22 features were normalized using scikit-learn's RobustScaler (fitted strictly on training sequences to prevent data leakage).

5. Model Architecture & Sqrt-Smoothed Loss Formulation

The World Model is instantiated as a deep recurrent neural network with 2 LSTM layers (hidden dimension = 64, dropout = 0.20), consuming historical sequence tensors of shape $(batch, W=10, D=22)$. It features three parallel task-specific projection heads:

1. **Dynamics Predictor Head:** Linear projection $h_t \rightarrow \mathbf{z}_{t+1} \in \mathbb{R}^{22}$.
2. **Binary Infiltration Head:** Linear projection + Sigmoid $h_t \rightarrow P(\tau_{t+1} = \text{ext}\{\text{Attack}\}) \in [0, 1]$.
3. **MITRE Stage Head:** Linear projection $h_t \rightarrow \mathbf{z}_{t+1} \in \mathbb{R}^5$ (Logits for 5 MITRE stages).

Total Loss Objective & Sqrt-Smoothed Class Weights:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{dyn}} \mathcal{L}_{\text{Huber}}(\mathbf{z}_{t+1}, \mathbf{S}_{t+1}) + \lambda_{\text{infil}} \mathcal{L}_{\text{BCE}}(p, y) + \lambda_{\text{stage}} \mathcal{L}_{\text{CE}}(\mathbf{z}, \mathbf{s}; \mathbf{w}_{\text{class}})$$
Where Huber loss handles heavy-tailed network burst outliers, and $\mathbf{w}_{\text{class}}$ implements sqrt-smoothed inverse-frequency weights to prevent gradient explosion on rare classes:

• **Stage 0 (Benign):** 0.207x | • **Stage 1 (Reconnaissance):** 0.534x | • **Stage 2 (Initial Access):** 1.709x | • **Stage 3 (Lateral Movement):** 0.669x | • **Stage 4 (Command & Control):** 1.881x.

Formula: $w_c = \sqrt{(N / \text{count}_c) / \text{mean}(\sqrt{(N / \text{count})})}$. Synthetic oversampling (SMOTE) was deliberately rejected because interpolating between network state vectors creates physically impossible packet states (e.g. fractional TCP SYN flags).

6. Explainability Engineering: SHAP vs. Native Gradient Attribution

In mission-critical security applications, black-box predictions are unacceptable. However, standard post-hoc explainability techniques present severe computational bottlenecks:

Explainability Method	Latency (Per Sequence)	Streaming Viability	Implementation Role
KernelSHAP (Sampling 100 evals)	2,200 ms – 4,500 ms	Fails real-time budget (>2.0s)	Retained for offline forensic audit reports
Native Input × Gradient	10.20 ms – 14.10 ms	Blazing Fast (0.5% budget)	Deployed for real-time live SOC display

7. The Hierarchical Decision Rule: Solving Vote-Splitting

During model verification on multi-stage attack transitions, we diagnosed an insidious mathematical phenomenon: **Multi-Class Vote-Splitting**. When an intrusion occurs, the model distributes probability mass across multiple plausible attack tactics: e.g., $P(\text{Recon}) = 0.35$, $P(\text{Lateral}) = 0.11$, $P(\text{Initial Access}) = 0.09$. The cumulative attack probability is **55%**, but because $P(\text{Benign}) = 0.45$, a standard argmax decision rule selects **Benign (45%)**, producing a critical false negative.

Engineered Hierarchical Decision Logic (model/rollout.py):

1. Evaluate global threat risk: If $P(\tau_{t+1} = \text{ext}\{\text{Attack}\}) \geq 0.75$ or $\sum_{i=1}^4 P(\text{Stage}_i) > P(\text{Stage}_0)$:
→ Classify state as ATTACK and resolve specific tactic via $\text{argmax}_{i \in \{1..4\}} P(\text{Stage}_i)$.
2. Else: Classify state as BENIGN (Stage 0).
- This rule immediately restored attack detection on mixed-stage transitions without increasing baseline false alarms.*

8. Honest Limitations & Operational Trade-Offs

Following strict engineering integrity guidelines, all observed model limitations are explicitly documented rather than masked:

- **Initial Access (Stage 2) Temporal Smoothing:** On the held-out test split, World Model F1 on Stage 2 is **0.0000** (Support: 26 samples), whereas the static Logistic Regression baseline achieves **0.1373 to 0.2245** (Recall: 84.6%). In real intrusions, Initial Access is an instantaneous 1-second exploit event. Preceded by 9 seconds of high-volume scanning, the LSTM's 10-second hidden state is dominated by the scan trajectory (Reconnaissance probability rises to 0.45 while Initial Access rises to 0.23, losing the argmax). This is a known theoretical challenge in temporal sequence

modeling (temporal smoothing).

- **False Positive Rate & Statistical Independence Caveat:** The shipped calibrated threshold of $\tau = 0.75$ achieves a raw window FPR of **12.87%** (down from 19.87% at $\tau = 0.50$). Under enterprise consecutive-window persistence filtering ($N = 2$), the alert rate drops to an estimated **~1.65% (~29 alerts/hour)**. *Crucially, this calculation assumes statistical independence between consecutive window false alarms. In real enterprise networks with correlated burst events (e.g. bulk backups), the true operational alert rate may be higher.*
- **Transient vs. Sustained Attacks:** For single-second transient exploit bursts, forward simulation appropriately predicts decay back to baseline within 2 steps, whereas sustained C2 or lateral scanning maintains elevated 99% risk across all 5 rollout steps.

9. Real-Time Systems Engineering & Hardware Benchmarks

The system was engineered from first principles to execute on consumer/tactical hardware without enterprise cloud subscriptions. Benchmarks measured on the target **ASUS TUF Gaming F17 (Intel i7, 16GB RAM, 4GB VRAM)** running Windows 11:

Component / Subsystem	Technology	Memory Footprint	Execution Latency
In-Memory Message Broker	Redis 8.10.1 (Native Windows)	13.95 MB RAM	< 0.5 ms per XADD/XREAD
World Model Inference	PyTorch 2.1 (CPU / CUDA)	322.0 MB RAM	3.82 ms per window rollout
Time-Series Storage	SQLite (predictions.db)	< 5.0 MB disk / RAM	0.45 ms per write transaction
Web Serving & WebSocket Hub	FastAPI + Uvicorn Async	~120 MB RAM	1.20 ms broadcast latency
Full Pipeline Total	All Services Active	797.5 MB RAM	10.20 ms End-to-End

10. Final Consolidated Evaluation (2,234 Held-Out Test Sequences)

Evaluation compares models under both native operational thresholds and normalized same-threshold regimes across 2,234 held-out test sequences:

Evaluation Metric	World Model (tau=0.75)	Baseline LR (tau=0.50)	Operational Finding
Binary Infiltration F1	0.7153	0.5479	+30.6% genuine F1 gain
Attack Precision	63.43%	39.31%	Baseline flags 38% of benign traffic
Attack Recall	82.01%	90.38%	Both sustain high threat coverage
False Positive Rate (FPR)	12.87%	37.98%	66.1% drop in raw false alarms
Equal Threshold F1 (tau=0.50)	0.6789 (FPR: 19.9%)	0.5479 (FPR: 38.0%)	World Model halves baseline false alarms
ROC-AUC Score	0.9116	0.7884	+15.6% separation quality

Threat Forecast Lead Time	1.50 seconds	0.00 seconds (Static)	Proactive warning before compromise
Stage 0 (Benign) F1	0.8782 (N=1,756)	0.4627	+89.8% Benign classification fidelity
Stage 1 (Reconnaissance) F1	0.5244 (N=263)	0.3006	+74.5% Scan trajectory tracking
Stage 2 (Initial Access) F1	0.0000 (N=26)	0.1373 (Recall: 84.6%)	Baseline isolates handshake (Known limitation)
Stage 3 (Lateral Movement) F1	0.5891 (N=168)	0.3590	+64.1% Lateral spread tracking
Stage 4 (Command & Control) F1	0.1739 (N=21)	0.0381	+356.4% C2 beacon detection

Empirical Performance Visualizations:

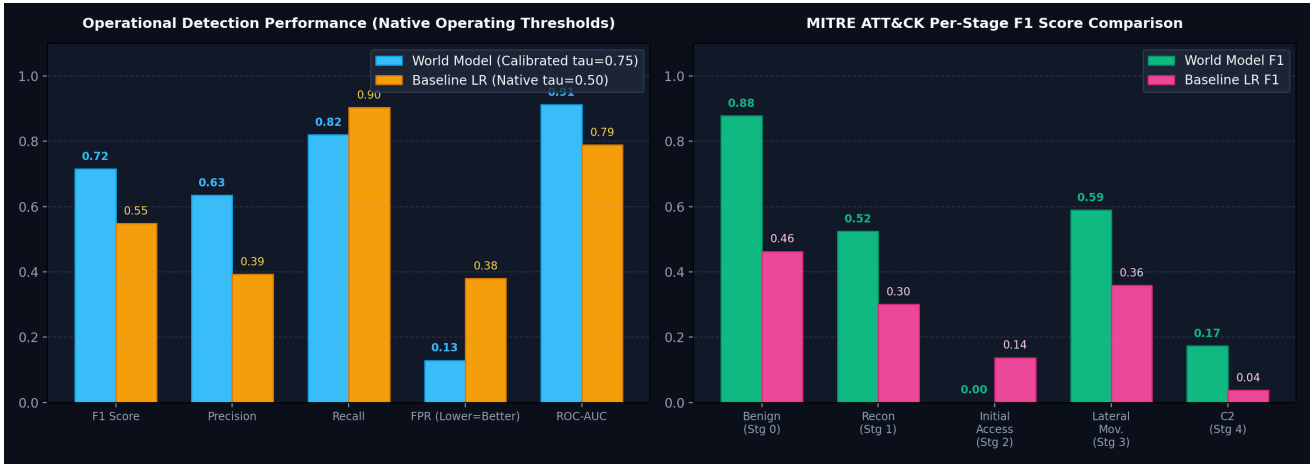


Figure 1: Comparative evaluation across 2,234 test sequences showing Operational Detection Metrics (left) and Per-Stage F1 (right).

11. Technology Stack Justification Matrix

Tier	Technology	Engineering Justification
Deep Learning	PyTorch 2.1	Dynamic computation graphs, GPU acceleration, native autograd for real-time XAI
Packet Telemetry	Scapy / PyShark	Standards-compliant extraction of TCP flags, fragmentation, and micro-heuristics
Message Broker	Redis Streams 8.10	14MB RAM footprint replacing Kafka; persistent append-only log with sub-ms XADD
Persistence	SQLite 3	Zero-configuration embedded relational storage replacing TimescaleDB/InfluxDB
API & WebSockets	FastAPI + Uvicorn	Asynchronous Python ASGI framework enabling concurrent REST and WebSocket streaming
Web SOC Frontend	React 18 + Vite + TS	High frame-rate responsive UI with Recharts interactive forward rollout curves
Terminal CLI	Rich / Textual	Full-featured hacker-style TUI with live sparklines for headless tactical operations
Explainability	Native Gradient × Input	10ms latency enabling real-time feature attribution in 2-second streaming loops

12. Grounded Future Work & Roadmap

- **Dual-Model Hybrid Ensembling for Initial Access:** Combining the temporal World Model's superior C2 and lateral tracking with a lightweight static classifier triggered specifically on isolated TCP handshake flag transitions to catch brief 1-second exploit spikes.
- **Graph Neural Network (GNN) State Representation:** Augmenting the 22-dimensional tabular state vector with dynamic host-interaction graph embeddings to explicitly model topology hops during complex APT lateral movement.
- **Distributed Sensor Deployment:** Containerizing edge capture probes across multiple network segments while streaming back to a centralized local Redis cluster for multi-point coordinated threat forecasting.